```python
"""
Console rendering utilities for the Weather Dashboard.
Produces tables and ASCII bar-charts directly in the terminal.

"""

from models import WeatherRecord


# Separator helper

def _separator(width: int = 60, char: str = "-") -> str:
    return char * width


# Current weather panel

def print_current_weather(record: WeatherRecord) -> None:
    """Print a formatted panel for the current weather."""
    print()
    print(_separator(60, "="))
    print(f"  {record.city}, {record.country}  —  {record.recorded_date}")
    print(_separator(60, "="))
    print(f"    Temperature : {record.temperature_c}°C  ({record.temperature_f}°F)")
    print(f"    Feels like  : {record.feels_like_c}°C")
    print(f"    Humidity    : {record.humidity_pct}%")
    print(f"    Wind speed  : {record.wind_speed_kmh} km/h")
    print(f"    Condition   : {record.weather_description}")
    print(_separator(60, "="))
    print()
```

```python


# 7 days forecast table

def print_forecast_table(forecast: list[WeatherRecord]) -> None:
    """Print a table 7-day forecast."""
    print(_separator(60))
    print(f"  {'DATE':<12} {'CONDITION':<22} {'TEMP (°C)':>9} {'WIND km/h':>10}")
    print(_separator(60))
    for rec in forecast:
        print(
            f"  {str(rec.recorded_date):<12} {rec.weather_description:<22} "
            f"{rec.temperature_c:>9.1f} {rec.wind_speed_kmh:>10.1f}"
        )
    print(_separator(60))
    print()


# ASCII temperature bar chart

def print_temperature_chart(forecast: list[WeatherRecord]) -> None:
    """
    Render a horizontal ASCII bar chart of daily temperatures.

    """
    if not forecast:
        print("  No forecast data to chart.")
        return

    temps: list[float] = [r.temperature_c for r in forecast]
    dates: list[str] = [str(r.recorded_date)[5:] for r in forecast]

    min_temp = min(temps)
```

```python
    max_temp = max(temps)
    temp_range = max_temp - min_temp if max_temp != min_temp else 1.0

    bar_max_width = 30

    print()
    print("    Temperature Trend (7-day)")
    print(_separator(55))
    for day_label, temp in zip(dates, temps):
        bar_len = int(((temp - min_temp) / temp_range) * bar_max_width)
        bar_len = max(bar_len, 1)
        bar = "" * bar_len
        print(f"  {day_label}  {bar:<31} {temp:>5.1f}°C")
    print(_separator(55))
    print(f"  Range: {min_temp}°C – {max_temp}°C")
    print()


# Favorites list

def print_favorites(favorites: list) -> None:
    """Print saved favorite cities."""
    print()
    print(_separator(55))
    print("    Saved Favorite Cities")
    print(_separator(55))
    if not favorites:
        print("  (no favorites saved yet)")
    for i, city in enumerate(favorites, start=1):
        print(f"  {i}. {city}")
    print(_separator(55))
    print()


# Weather log summary

def print_log_summary(rows: list[dict]) -> None:
    """Print a short summary of the weather log CSV
```

```
."""
    print()
    print(_separator(55))
    print(f"    Weather Log — {len(rows)} entries")
    print(_separator(55))
    if not rows:
        print("  (log is empty)")
    else:
        recent = rows[-5:]
        for row in recent:
            print(
                f"  {row['date']}  {row['city']:<15}"
                f"{float(row['temperature_c']):>6.1f}°C  {row['description']}"
            )
    print(_separator(55))
    print()
```